

Softwarekonstruktion WS 15/16 – Übung 1

Ausgabe: 26.10.2015, Abgabe: 09.11.2015 08:00 Uhr

Organisatorisches

Präsenzübungen

- Kurzvorstellung wesentlicher Fallstricke aus den Hausübungen.
- Bearbeitung der Präsenzübungen in Gruppen. Die Tutoren stehen für Fragen bei der Bearbeitung zur Verfügung, es ist aber **keine** Vorrechenübung.
- Gemeinsames Besprechen der Präsenzübungen.

Hausübungen

- Es wird insgesamt **6 Hausübungen à 40 Punkte** geben.
- Deadlines der Abgaben sind auf den Hausübungszetteln angegeben (in der Regel zwei Wochen nach Ausgabe).
- Abgabe der Hausübungen:
 - in Gruppen bis maximal 4 Personen einer Übungsgruppe
 - Einwurf in die Briefkästen Nr. 21 in der Otto-Hahn-Str. 12 oder persönlich in der Übungsgruppe
 - Auf der Abgabe sind Name, Matrikelnummer und die Übungsgruppe aller beteiligten Personen zu vermerken
 - **keine Abgabe** per E-Mail, Hauspost, persönliche Abgabe an Lehrstuhl oder Sekretariat
- Rückgabe der Hausübungen:
 - in der entsprechenden Übung
 - nicht abgeholte Übungszettel liegen im Sekretariat zur Abholung bereit
- Einige Aufgaben der Hausübungen werden online in Moodle bearbeitet. Um die Online-Übungen zu bearbeiten, gehen Sie wie folgt vor:
 - Rufen Sie <https://moodle2.tu-dortmund.de> auf
 - Melden Sie sich mit Ihrem Unimail-Account an (SSO)
 - Schreiben Sie sich in den Kurs „Softwarekonstruktion, LSF, 040211“ ein.
 - Folgen Sie den Anweisungen in Moodle
 - Bei technischen Problemen, wenden Sie sich per E-Mail an Ihren Tutor
- Abgabe von Duplikaten führt zu 0 Punkten für alle beteiligten Gruppen

Studienleistung

- Insgesamt sind mind. **60 Punkte (=50%) aus Hausübung 1-3** und mind. **60 Punkte (=50%) aus Hausübung 4-6** zu erreichen.

Einführung in OCL

1.1 OCL Begriffe

Erläutern Sie die Begriffe Einschränkung, Klasseninvariante, Vorbedingung und Nachbedingung.

Einschränkung: OCL ist eine formale Sprache zur Definition von Einschränkungen auf einem oder mehreren Teilen eines UML-Models.

Klasseninvariante: Eine Bedingung, die (fast) immer von allen Instanzen einer Klasse erfüllt werden muss.

Vorbedingung: Eine Bedingung, die erfüllt sein muss, bevor eine Operation ausgeführt werden kann.

Nachbedingung: Eine Bedingung, die nach dem Ausführen einer Operation erfüllt sein muss.

1.2 Sammlungen in OCL

Erläutern Sie stichpunktartig die Unterschiede zwischen **Set**, **Bag**, **OrderedSet** und **Sequence**.

SET: ungeordnete Sammlung ohne Duplikate

BAG: ungeordnete Sammlung, evtl. mit Duplikaten

ORDEREDSET: geordnete Sammlung ohne Duplikate

SEQUENCE: geordnete Sammlung, evtl. mit Duplikaten

1.3 Vordefinierte Operationen

Die OCL ist eine sehr mächtige Sprache, daher wird in der Veranstaltung eine dedizierte Teilmenge der in OCL vordefinierten Operationen betrachtet. Machen Sie sich mit den nachfolgenden Operationen vertraut, indem Sie stichpunktartig beschreiben, was die jew. Operationen berechnen.

- **logische Operatoren**

- and, or, xor Konjunktion (und), Disjunktion (oder) sowie Kontravalenz
- not, implies ——— **not: logische Negation, implies: logische Implikation**
- drücken Sie implies nur mit Hilfe der anderen logischen Operatoren aus: _____
 a implies b $\equiv$ not a or b. Achtung bei Wahrheitstabelle: nur a=true, b=false liefert (a implies b) = false

- **Aufzählungstypen (enumeration types)**

- *variable = enumeration name :: enumeration value* Enumerations sind Daten-

typen der UML, welche die Definition einer Reihe von Literalen als mögliche Werte erlauben. Für den Zugriff auf die Werte ist die :: Syntax zu verwenden.

• Operationen auf Objekten

1. boolesche Operatoren (wenn *self* mit Objekt *o* vergleichbar ist)

- $=(o:T): \text{Boolean}$ gibt true zurück, wenn *self* gleich dem Objekt *o* ist
- $<>(o:T): \text{Boolean}$ gibt true zurück, wenn *self* ungleich dem Objekt *o* ist
- $<(o:T): \text{Boolean}$ gibt true zurück, wenn *self* kleiner als *o* ist
- $<=(o:T): \text{Boolean}$ gibt true zurück, wenn *self* kleiner gleich *o* ist
- $>(o:T): \text{Boolean}$ gibt true zurück, wenn *self* größer als *o* ist
- $>=(o:T): \text{Boolean}$ gibt true zurück, wenn *self* größer gleich *o* ist

• Operationen auf Collections

1. einfache Operationen

- $\rightarrow \text{size}(): \text{Integer}$ **gibt die Anzahl aller Elemente der Collection zurück**
- $\rightarrow \text{sum}(): \text{Real}$ **addiert alle Elemente der Collection**
- $\rightarrow \text{isEmpty}(): \text{Boolean}$ **gibt true zurück, wenn Collection leer ist**
- $\rightarrow \text{notEmpty}(): \text{Boolean}$ gibt true zurück, wenn *Collection* mindestens ein Element enthält

2. Elementbezogene Operationen

- $\rightarrow \text{includes}(o:T): \text{Boolean}$ _____ **gibt true zurück, wenn *o* in Collection enthalten ist**
- $\rightarrow \text{excludes}(o:T): \text{Boolean}$ gibt true zurück, wenn *o* nicht in der *Collection* enthalten ist
- $\rightarrow \text{count}(o:T): \text{Integer}$ _____ **zählt, wie oft *o* in der Collection enthalten ist**
- $\rightarrow \text{isUnique}(\text{expr}: \text{OclExpression}): \text{Boolean}$ _____ **gibt true zurück, wenn alle Objekte nach Anwendung der *expr* einzigartig in der Collection enthalten sind**

3. boolesche Operatoren

- $= (c: \text{Collection}(T)): \text{Boolean}$ **gibt true zurück, wenn *self* genau die und nur die Elemente der Collection *c* enthält** _____

- $\langle \rangle (c: \text{Collection}(T))$: Boolean gibt true zurück, wenn *self* nicht genau die bzw. nicht nur die Elemente der Collection *c* enthält _____

4. Collection-Operationen

- $\text{aSet} \rightarrow \text{union}(s: \text{Set}(T)): \text{Set}(T)$ _____
Die Vereinigung einer ungeordneten Menge ohne Duplikate (Set) mit einer ungeordneten Menge ohne Duplikate (Set) liefert eine ungeordnete Menge ohne Duplikate (Set) mit dem selben Typ (Achtung! Nur auf Bag und Set spezifiziert. Siehe Abschnitt 11.7.2 in OCL 2.4 specs und Tabelle weiter unten)
- $\text{aBag} \rightarrow \text{intersection}(b: \text{Bag}(T)): \text{Bag}(T)$ Der Schnitt einer ungeordneten Menge mit Duplikaten (bag) mit einer Bag liefert eine Bag, welche den selben Typ T hat!
- $\rightarrow \text{includesAll}(c: \text{Collection}(T))$: Boolean _____
gibt true zurück, wenn *self* alle Elemente der Collection *c* enthält
- Nicht jede Collection-Operation kann auf jeder der vier Collections (Set, Bag, OrderedSet, Sequence) angewendet werden. Tragen Sie in die unten stehenden Tabellen ein, welche Operation auf welche Art von Collection angewendet werden kann. Schauen Sie gegebenenfalls in der OCL Spezifikation nach (Abschnitt 11.7).

Es gilt zum Beispiel im Allgemeinen **nicht**

* $\text{aCollection} \rightarrow \text{union}(c: \text{Collection}(T)): \text{Collection}(T)$

für jede beliebige Collection. Aber zum Beispiel

* $\text{aBag} \rightarrow \text{union}(s: \text{Set}(T)): \text{Bag}(T)$.

Füllen Sie die restlichen Felder aus. Ist eine Kombination nicht möglich, so tragen Sie ein X ein.

union	Set	Bag	Sequence	OrderedSet
aSet	Set			X
aBag	Bag			
aSequence				
aOrderedSet				

union	Set	Bag	Sequence	OrderedSet
aSet	Set	Bag	X	X
aBag	Bag	Bag	X	X
aSequence	X	X	Sequence	X
aOrderedSet	X	X	X	X

intersection	Set	Bag	Sequence	OrderedSet
aSet				
aBag		Bag		
aSequence				
aOrderedSet				

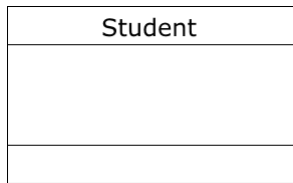
intersection	Set	Bag	Sequence	OrderedSet
aSet	Set	Set	X	X
aBag	Set	Bag	X	X
aSequence	X	X	X	X
aOrderedSet	X	X	X	X

5. Iterator-Ausdrücke

- $\rightarrow \text{select}(\text{expr}:\text{OclExpression}): \text{Collection}(\text{T})$ _____
_____ gibt
alle Elemente der C. zurück, welche die *expr* erfüllen
- $\rightarrow \text{collect}(\text{expr}:\text{OclExpression}): \text{Collection}(\text{T})$ _____
_____ gibt
die Collection zurück, nachdem die *expr* auf alle Elemente angewandt wurde
- $\rightarrow \text{reject}(\text{expr}:\text{OclExpression}): \text{Collection}(\text{T})$ _____ liefert eine Collection, die
alle Elemente von *self* enthält, außer diejenigen, welche die *expr* erfüllen
- $\rightarrow \text{forAll}(\text{expr}:\text{OclExpression}):\text{Boolean}$ _____
gibt true zurück, wenn alle Elemente der Collection die *expr* erfüllen

• Operationen auf Klassen

- $\text{classifier.allInstances}(): \text{Set}(\text{T})$
_____ gibt ein Set zurück, welches alle Instanzen des classifiers enthält



Formulieren Sie folgende Aussage zu der Klasse *Student* mit Hilfe der OCL:

Es soll nicht mehr als 10.000 Studenten geben.

context Student

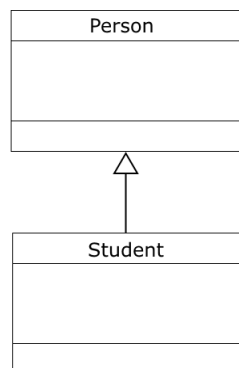
inv: Student.allInstances()->size() <= 10000

• weitere Operationen auf Objekten

1. Typ vs. Art

- self.oclIsTypeOf(*typespec*): Boolean _____
_____ gibt true zurück, wenn self dem Typen *typespec* entspricht
- self.oclIsKindOf(*typespec*): Boolean _____
_____ gibt true zurück, wenn self dem Typen *typespec* oder einem Subtypen von *typespec* entspricht

Kreuzen Sie alle gültigen Beziehungen des folgenden Klassendiagramms an!



	oclIsTypeOf		oclIsKindOf	
	Person	Student	Person	Student
Person				
Student				

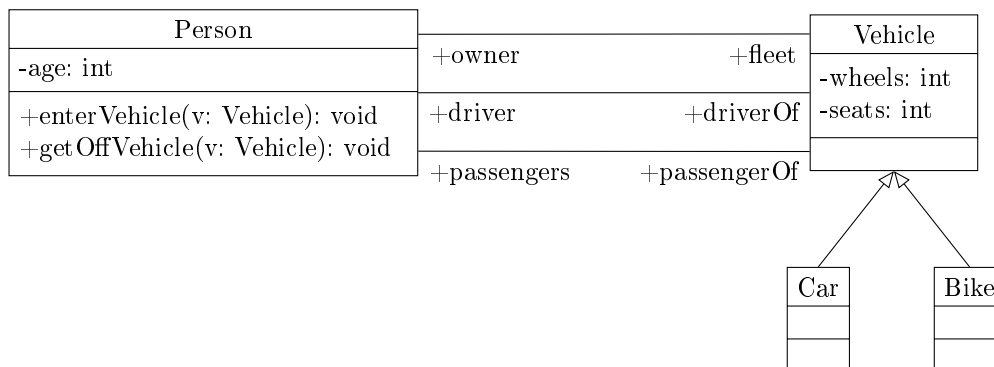
	oclIsTypeOf		oclIsKindOf	
	Person	Student	Person	Student
Person	X		X	
Student		X	X	X

2. undefiniert vs. ungültig

- self.oclIsUndefined(): Boolean _____
_____ gibt true zurück, wenn self invalid (ungültig) oder null ist
- self.oclIsInvalid(): Boolean _____
_____ gibt true zurück, wenn self invalid (ungültig) ist

1.4 OCL II

Erweitern sie das folgende Person-Vehicle-Beispiel. Benutzen Sie dafür soweit möglich OCL-Bedingungen:



- Das Alter einer Person muss positiv sein.

```
context Person
inv: self.age > 0
```
- Autos haben 4 Räder, Fahrräder haben 2 Räder.

```
context Car
inv: self.wheels = 4
context Bike
inv: self.wheels = 2
```
- Der Besitzer einer Autos muss mindestens 18 Jahre alt sein.

```
context Car
inv: self.owner.age >= 18
```
- Neben der Anzeige des Eigentums soll auch abbildbar sein, wer mit dem Auto fährt. Dabei ist zwischen Fahrer und Mitfahrenden zu unterscheiden. Es darf immer nur einen Fahrer geben. Zudem darf die Anzahl von Fahrer und Mitfahrern die Anzahl vorhandener Sitzplätze nicht überschreiten. Mitfahrer können nur mitfahren wenn es auch einen Fahrer gibt. Eine Person kann zu einem Zeitpunkt immer nur in einem Fahrzeug sitzen. Ist sie der Fahrer muss ihr das Fahrzeug gehören.

```
context Car
inv: self.driver->size() <= 1
inv: self.passengers->size() <= self.seats-1
inv: (not self.passengers->isEmpty()) implies self.driver->size() = 1
context Person
inv: self.driverOf->union(self.passengerOf)->size() <= 1 context Vehicle
inv: self.driver = self.owner
```
- Es sollen Operationen zur für das Ein- und Aussteigen modelliert werden. Hierbei soll auf folgendes geachtet werden: Die einsteigende Person darf nicht bereits in einem anderen Fahrzeug sitzen. Eine Person kann nur in ein Fahrzeug einsteigen, wenn noch ausreichend Platz im Fahrzeug ist. Nach dem Einsteigen ist die Person entweder Fahrer oder Passagier des Fahrzeugs. Eine aussteigende Person muss in einem Fahrzeug sitzen. Nach dem Aussteigen ist die Anzahl der im Fahrzeug sitzenden Personen um 1 kleiner.

```
context Person::enterVehicle(v:Vehicle):void
pre: self.driverOf->union(self.passengerOf)->isEmpty()
pre: v.owner <> self implies v.seats - v.passengers->size() > 0
```

```
pre: v.owner = self implies v.driver->isEmpty()
post: v.owner = self implies v.driver->includes(self)
post: v.owner <> self implies v.passengers->includes(self)
context Person::getOffVehicle(v: Vehicle):void
pre: v.driver->union(v.passengers)->includes(self)
post: v.driver@pre->union(v.passengers@pre)->size()=
v.driver->union(v.passengers)->size()+1
```

Präsenzübung

1.5 Sammlungen in OCL

Geben sie jeweils eine Lösung für die nachfolgenden OCL-Ausdrücke an. Sollte ein Ausdruck keine gültige Anfrage sein, so notieren Sie dies.

Schlagen sie bei Bedarf die entsprechende Stelle in der OCL Spezifikation nach:

<http://www.omg.org/spec/OCL/2.4/PDF/>

Beispiel: $\text{Set}\{1,4,7,10\} - \text{Set}\{4,7\} = \text{Set}\{1,10\}$

- $\text{Set}\{1,2,3,4,5\} \rightarrow \text{sum}() = 15$
- $\text{Set}\{1,2,3,4,5\} \rightarrow \text{size}() = 5$
- $\text{Sequence}\{'h','a','l','l','o'\} \rightarrow \text{last}() = 'o'$
- $\text{Bag}\{'h','a','l','l','o'\} \rightarrow \text{last}() = \text{keine gültige Anfrage}$
- $\text{Set}\{'h','a','l','l','o',' ','w','e','l','t'\} \rightarrow \text{one}(e|e='a') = \text{true}$
- $\text{Set}\{'h','a','l','l','o',' ','w','e','l','t'\} \rightarrow \text{select}(e|e='l') \rightarrow \text{size}() = 1$
- $\text{Set}\{1,2,-1,-2\} \rightarrow \text{isUnique}(e|e*e) = \text{false}$

1.6 Veranstaltungshallen I

Betrachten Sie das UML-Klassendiagramm über Veranstaltungshallen, wie beispielsweise die Dortmunder Westfalenhalle, in nachfolgender Abbildung 1.

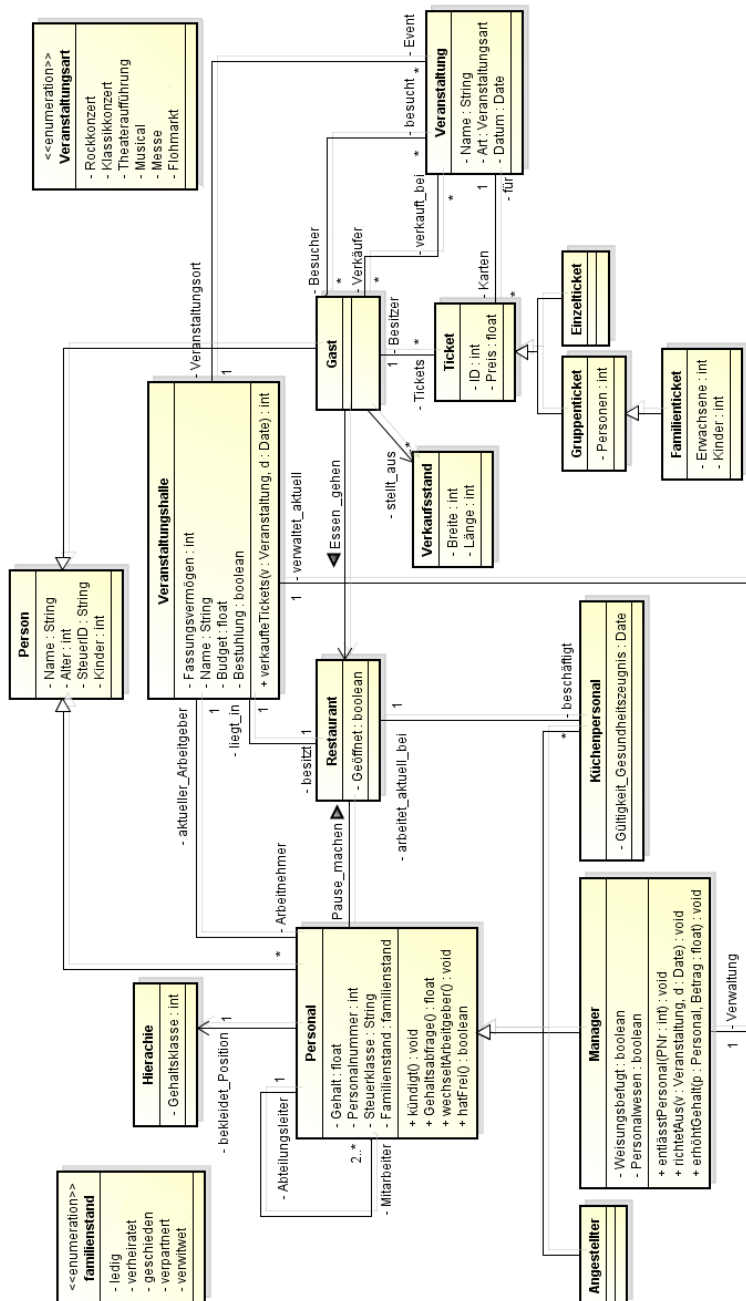


Abbildung 1: UML-Klassendiagramm der Veranstaltungshallen

Das UML-Klassendiagramm in Abbildung 1 erlaubt leider noch die Konstruktion ungewollter Konstellationen. Daher muss die OCL verwendet werden, um notwendige Randbedingungen durch die Spezifikation von Einschränkungen (Constraints) zu formulieren.

Drücken Sie folgende Constraints in Form von gültigen OCL-Ausdrücken aus

Formulieren Sie in dem Kontext, den die Aufgabenstellung impliziert. Verwenden Sie dabei ausschließlich die Operationen aus dem Einführungsteil dieses Übungsblatts.

1. Das Alter eines Menschen ist immer positiv.
`context Person`
`inv: self.Alter > 0`
2. Eine Veranstaltung können maximal so viele Gäste besuchen, wie der entsprechende Veranstaltungsort fassen kann.
`context Veranstaltung`
`inv: self.Besucher -> size() <= self.Veranstaltungsort.Fassungsvermögen`
oder
`inv: self.Veranstaltungsort.Fassungsvermögen >= self.Besucher -> size()`
3. In der Veranstaltungshalle *Westfalahalle* muss es für jede Veranstaltung mindestens 60 Karten geben.
`context Veranstaltungshalle`
`inv: self.Name= 'Westfalahalle' implies self.Event ->`
`forall(v:Veranstaltung|v.Karten-> size() >= 60)`
4. Aus wirtschaftlichen Gründen muss die *Westfalahalle* sparen. Daher dürfen die Gehälter aller Arbeitnehmer der Westfalahalle 60% des Budgets der Halle nicht übersteigen.
`context Veranstaltungshalle`
`inv: self.Name= 'Westfalahalle' implies self.Arbeitnehmer ->`
`collect(p:Personal|p.Gehalt) -> sum() <= self.Budget * 0.6`
5. Bei einem Flohmarkt darf es in der Veranstaltungshalle keine Bestuhlung geben.
`context Veranstaltung`
`inv: self.Art= Veranstaltungsart::Flohmarkt implies`
`self.Veranstaltungsort.Bestuhlung = false`
oder
`inv: self.Art= Veranstaltungsart::Flohmarkt implies`
`self.Veranstaltungsort.Bestuhlung <> true`

Hausübung

Hinweis zu dieser Hausübung: Die Online-Übungen 1.7 und 1.8 müssen von jedem Teilnehmer in Moodle durchgeführt und abgeschlossen werden, können jedoch in Gruppen besprochen werden. Aufgabe 1.9 muss auf Papier in den Briefkasten 21, OH 12 eingeworfen werden (Gruppenabgabe: bis zu vier Personen einer Übungsgruppe).

1.7 Online-Übung: OCL-Ausdrücke (10 Punkte)

Diese Übung umfasst 10 Fragen, bei denen das Ergebnis von OCL-Ausdrücken bestimmt werden muss. Bitte folgen sie hierzu den Anweisungen zu den Online-Übungen auf Seite 1 dieses Übungsblatts.

1.8 Online-Übung: OCL-Invarianten (10 Punkte)

Diese Übung umfasst 5 Fragen, bei denen zu einem gegebenen Klassendiagramm klartextlich formulierte Constraints mit OCL-Invarianten umgesetzt werden müssen. Bitte folgen sie hierzu den Anweisungen zu den Online-Übungen auf Seite 1 dieses Übungsblatts.

1.9 Veranstaltungshallen II (20 Punkte)

Betrachten Sie das UML-Klassendiagramm zu Veranstaltungshallen aus Aufgabe 1.6. Drücken Sie folgende Constraints in Form von korrekten OCL-Ausdrücken aus.

Verwenden Sie dabei ausschließlich die Operationen aus dem Einführungsteil dieses Übungsblatts.

1. Manager verdienen mehr als 5.000 Euro, Angestellte und Küchenpersonal jedoch maximal 2500 Euro.

```
context Manager
inv: self.Gehalt > 5000
context Angestellter
inv: self.Gehalt <= 2500
context Küchenpersonal
inv: self.Gehalt <= 2500
```

2. Jedes Ticket besitzt eine eindeutige ID.

```
context Ticket
inv: self.allInstances() -> isUnique(t:Ticket|t.ID)
alternativ:
inv: self.allInstances() -> forAll(t1,t2:Ticket|t1 <> t2 implies t1.ID <> t2.ID)
```

3. Es darf nur bei Flohmärkten Verkäufer geben, bei allen anderen Veranstaltungen sind alle Gäste Besucher. Zudem muss es bei Flohmärkten mindestens einen Verkäufer geben.

```
context Veranstaltung
inv: (self.Art <> Veranstaltungsart::Flohmarkt implies self.Verkäufer ->
isEmpty()) and
(self.Art = Veranstaltungsart::Flohmarkt implies self.Verkäufer -> notEmpty())
```

4. Bei Flohmärkten dürfen pro Verkäufer in der *Westfalenhalle* nicht mehr als 5 Verkaufsstände aufgestellt werden.

```
context Veranstaltung
inv: (self.Art= Veranstaltungsart::Flohmarkt and self.Veranstaltungsort.Name=
```

- ```
'Westfalenhalle') implies
self.Verkäufer -> forAll(g:Gast|g.stellt_aus -> size() <= 5)
```
5. Eine Theatervorführung muss mehr als 10 Besucher haben (um nicht abgesagt zu werden). Zudem darf dann in der Veranstaltungshalle die Bestuhlung nicht fehlen.
 

```
context Veranstaltung
inv: self.Art= Veranstaltungsart::Theatervorführung implies
(self.Besucher -> size() > 10 and self.Veranstaltungsort.Bestuhlung= true)
alternativ:
inv: self.Art= Veranstaltungsart::Theatervorführung implies (self.Besucher -> size()
> 10 and self.Veranstaltungsort.Bestuhlung <> false)
```
  6. Kultur soll bereits in der Jugend gefördert werden. Aus diesem Grund müssen Kinder unter 6 Jahren keinen Eintritt zu Klassikkonzerten oder Musicals zahlen, d.h. Tickets dieser Veranstaltungskategorien sind für sie kostenlos.
 

```
context Ticket
inv: (self.Besitzer.Alter < 6 and (self.für.Art= Veranstaltungsart:: Musical
or self.für.Art= Veranstaltungsart:: Klassikkonzert)) implies self.Preis= 0.0
alternativ:
context Veranstaltung
inv:(self.Art= Veranstaltungsart::Klassikkonzert or self.Art= Veranstaltungsart::
Musical) implies self.Karten -> select(t:Ticket | t.Besitzer.Alter < 6) ->
forAll(t:Ticket |t. Preis= 0)
alternativ:
context Veranstaltung
inv:(self.Art= Veranstaltungsart::Klassikkonzert or self.Art= Veranstaltungsart::
Musical) implies self.Karten -> forAll(t:Ticket |t.Besitzer.Alter < 6 implies Preis=
0)
alternativ:
context Gast
inv: self.Alter > 6 implies self.Tickets -> select(t:Ticket |t.für.Art=
Veranstaltungsart::Klassikkonzert or t.für.Art= Veranstaltungsart::Musical)
-> forAll(t:Ticket | t.Preis = 0.0)
```
  7. Bei allen Veranstaltungen sind Gruppentickets teurer als Einzeltickets.
 

```
context Veranstaltung
inv: self.Karten -> forAll(t1,t2:Ticket| t1.ocllsKindOf(Gruppenticket)
and t2.ocllsKindOf(Einzelticket) implies t1.Preis > t2.Preis)
```
  8. Nachdem in einer Veranstaltungshalle eine Veranstaltung ausgerichtet wurde, werden von dem zuständigen Manager direkt 10 % der Ticketeinnahmen dem Budget der Veranstaltungshalle gutgeschrieben.
 

```
context Manager::richtetAus(v:Veranstaltung, d:Date): void
post: v.Veranstaltungsort.Budget = v.Veranstaltungsort.Budget@pre +
v.Karten -> collect (t:Ticket | t.Preis) -> sum() /10
alternativ:
post: self.verwaltet_aktuell.Budget = self.verwaltet_aktuell.Budget@pre +
v.Karten -> collect (t:Ticket | t.Preis) -> sum() * 0.1
```
  9. Nach dem Wechsel zu einem neuen Arbeitgeber verdient ein Manager aus dem Bereich Personalwesen 10 % mehr als bei seinem alten Arbeitgeber, wenn er bei dem neuen Arbeitgeber weisungsbefugt ist und in seiner neuen Abteilung mehr Mitarbeiter führen muss als in seiner alten. Zudem befindet er sich durch die Weisungsbefugnis in der nächsthöheren Gehaltsklasse.
 

```
context Manager::wechseltArbeitgeber(): void
post: (self.Weisungsbefugt = true and self.Personalwesen = true and
```

```
self.Mitarbeiter -> size() > self.Mitarbeiter@pre -> size()) implies
(self.Gehalt = self.Gehalt@pre *1.1 and
self.bekleidet_Position.Gehaltsklasse = self.bekleidet_Position@pre.Gehaltsklasse@pre+1)
```

10. Die Veranstaltungshalle kann den Ticketverkauf einer Veranstaltung zu einem bestimmten Datum abfragen. Dabei wird die Zahl aktuell verkaufter Tickets zurückgegeben.

```
context Veranstaltungshalle::verkaufteTickets(v:Veranstaltung, d:Date):int
post: result= v.Karten -> size()
alternativ:
post: v.Datum = d implies result= v.Karten -> size()
alternativ:
post: result= self.Event -> select(v1:Veranstaltung | v1 = v and v1.Datum= d).Karten
-> size()
```

Verwendung von `.` und `->`:

Zugriff auf Properties erfolgt mittels der `“.property”`-Notation, wobei nicht nur Attribute und Assoziationsenden als Properties betrachtet werden, sondern auch Operationen und Methoden, die seiteneffektfrei sind (bei denen `“isQuery=true”` gilt).

Der Zugriff auf Properties von Collections erfolgt mittels `“->property”`-Notation, das sind bspw. `size()`, `sum()`, `isEmpty()`, `notEmpty()`. Ausserdem werden alle Operationen auf Collections, die zum Einschränken der Collection benutzt werden mit `“arrow-syntax”` geschrieben, z.B. `select()`, `reject()`, `collect()`, `forAll()`, `exists()`, `closure()` und `iterate()`.